



Tribhuvan University
Institute of Science and Technology

A Final Year Project Report

On

College Management System

Submitted To

Department of Information Technology

Kathford International College of Engineering and Management

**In partial fulfillment of the requirement for the Bachelor Degree in Computer
Science and Information Technology**

Submitted By

Abhinav Singh Thapa (79010398)

Babita Subedi (79010404)

Khusi Dahal (79010410)

Under the Supervision of

Er Suwas Karki

SUPERVISER’S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Abhinav Singh Thapa, Babita Subedi** and **Khusi Dahal** entitled “**College Management System**” in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology be processed for the evaluation.

.....

Er Suwas Karki

Project Supervisor

Kathford International College of Engineering & Management

LETTER OF APPROVAL

This is to clarify that this project prepared by **Abhinav Singh Thapa (79010397)**, **Babita Subedi (79010404)** and **Khusi Dahal (79010438)** entitled “**College Management System**” in particular fulfillment of the requirement for the degree of B.Sc. in Computer Science and Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Er Suwas Karki

Supervisor/Faculty

Kathford International College

Balkumari, Lalitpur

Internal Examiner

Department of Information Technology

Kathford International College

Balkumari, Lalitpur

DHOD

Department of Information Technology

Kathford International College

Balkumari, Lalitpur

External Examiner

Tribhuvan University

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us throughout the completion of this project. This project would not have been possible without the guidance, encouragement, and cooperation of many individuals.

First and foremost, we would like to thank our project supervisor, **Er Suwas Karki**, for his valuable guidance, continuous support, constructive suggestions, and encouragement during every phase of the project. His knowledge and supervision greatly helped us in successfully completing this work.

We are also thankful to the faculty members of the Department of IT for providing us with the opportunity and necessary resources to carry out this project effectively.

We would like to express our heartfelt appreciation to our friends and classmates for their motivation, cooperation, and helpful suggestions throughout the development of the project.

We are deeply grateful to our parents and family members for their constant encouragement, moral support, and understanding during the entire period of this project.

Finally, we would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

ABSTRACT

The College Management System centralizes workflows such as student and teacher management, class attendance, fee collection, notices, and academic records into a single platform with secure role-based authentication. Tailored dashboards for administrators, teachers, and students ensure organized access to responsibilities and data. Core features include attendance tracking, fee reporting, subject allocation, performance analytics, and academic extensions like class teacher assignments. By reducing manual paperwork and offering predictive insights into student performance, the system enhances transparency, saves time, and supports better decision-making. Also, with the addition of prediction model, at-risk students, can be found out early. Prediction is done on the basics of 10 features while include the historic academic records, social condition of the student, and student behaviours. Random Forest is used as it works best for nonlinear relationships which matches final grade prediction. The system uses “Student-Performance-Prediction” dataset from Kaggle for training the random forest model. The system consists of 3 actors admin, teachers and students. Each role has different functionalities, with the admin having the highest priority. Agile methodology was used to complete the project where at each sprint different functionalities and modules were added to finish the project.

Keywords: attendance tracking, performance analytics, assignments, role-based authentication

TABLE OF CONTENT

SUPERVISER'S RECOMMENDATION	i
LETTER OF APPROVAL.....	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS	ix
1. INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.5 Development Methodology	3
1.6 Report Organization	3
2. BACKGROUND STUDY AND LITERATURE REVIEW	5
2.1 Background Study.....	5
2.2. Literature Review	6
3. SYSTEM ANALYSIS	7
3.1 System Analysis.....	7
3.1.1 Requirement Analysis	7
3.1.2 Feasibility Analysis.....	9
3.1.3 Analysis.....	10
4. SYSTEM DESIGN	15
4.1 Design.....	15
4.1.1 Database Design: Transformation of ER to Relations and Normalizations.....	15
4.2 Algorithm Details	19
5. IMPLEMENTATION	21
5.1 Implementation	21
5.1.1 Tools Used.....	21
5.1.2 Implementation Details of Modules	22
5.2 Testing	24
5.2.1 Test Cases for Unit Testing	24
5.2.2. Test Cases for System Testing	29

6.	CONCLUSION.....	32
6.1	Conclusion.....	32
6.2	Future Recommendation	32
7.	REFERENCES.....	33
8.	APPENDICES	34

LIST OF FIGURES

Figure 1.1	Agile methodology	3
Figure 3.1	Use Case Diagram of College Management System.....	8
Figure 3.2	DFD level 1 of College Management System..... Error! Bookmark not defined.	
Figure 3.3	Entity-Relationship Diagram.....	13
Figure 4.1	Relational Schema Diagram of administration and setup module	15
Figure 4.2	Relational Schema for Academic Performance and Daily Tracking Error! Bookmark not defined.	
Figure 4.3	Relational Schema for Fee module.....	18

LIST OF TABLES

Table 5.1	Unit Test UT-1: Prediction System Testing	24
Table 5.2	Unit Test UT-2: Authentication	26
Table 5.3	Unit Test UT-3: User Management	26
Table 5.4	Unit Test UT-4: Student management and class access	27
Table 5.5	Unit Test UT-5: Notices	28
Table 5.6	Unit Test UT-6: Assignment and Submission	28
Table 5.7	System Test ST-1: Admin End to End flow	29
Table 5.8	System Test ST-2: Teacher End to End flow	30
Table 5.9	SystemTest ST-3: Student End to End flow	31

LIST OF ABBREVIATIONS

API	Application Programming Interface
EDM	Educational Data Mining
CMS	College Management System
DFD	Data Flow Diagram
ER	Entity-Relationship
G1	Periodic Assessment Mark 1
G2	Periodic Assessment Mark 2
G3	Final Grade
IDE	Integrated Development Environment
JWT	JSON Web Token
npm	Node Package Manager
PK	Primary Key
RBAC	Role-Based Access Control
RF	Random Forest
ST	System Test
UI	User Interface
UT	Unit Test

1. INTRODUCTION

1.1 Introduction

Educational institutes rely primarily on manual methods for recording attendance and maintaining student records and performing student operations. The College Management System is a web application integrated with Machine Learning to manage student records and predict final grades along with school administration. By combining digital record-keeping with predictive analytics, this system transforms manual monitoring into a centralized web-based platform.

The system utilizes a multi-actor architecture involving Administrators, Teachers, and Students. Incorporating functionalities such as assignment distribution, notices, student record management, fee payment tracking streamlines the college workflow, replacing manual processes with a digital framework. Furthermore, role-specific dashboards unify data and insights for direct observation. Teachers can access student record summaries and academic trends, while administrators can monitor insights through graphs and charts.

To support academic monitoring, the system employs Random Forest models to predict final grades. The Random Forest model identifies the influential factors affecting student performance and provides data-driven insights for academic improvement. Predicting outcomes in advance enables the early identification of at-risk students, allowing for intervention prior to final assessments.

1.2 Problem Statement

Educational institutes rely on manual methods for student administration. Existing digital tools fragment academic workflows, such as assignment distribution and submission, and administrative tasks, such as fee processing, notices. This lack of integration creates operational inefficiencies and ambiguousness.

Furthermore, current systems typically function as static data repositories without incorporating predictive analytics. There is a need for a centralized platform that unifies academic and administrative modules while utilizing Machine Learning models to predict

student performance prior to final examinations. The project will implement a Data Visualization and Analytics Dashboard, enabling administrators to access student records and gain insights into attendance and performance, from centralized dashboard.

1.3 Objectives

The main objectives of this project are:

- To develop a web-based system that integrates student record management with school administration, streamlining both academic and administrative operations
- To forecast student final grades using machine learning models to provide early warnings and actionable insights prior to final exams.

1.4 Scope and Limitation

Scope:

- **Role-Based Dashboards:** The system provides Administrators, Teachers, and Students with secure, role-based access. Each actor utilizes a specific dashboard that visualizes relevant data, summaries, and system insights.
- **Management Modules:** Includes student profile management, attendance tracking, an assignment module for distribution and submission, and a fee payment tracking system.
- **Predictive Analytics:** Implement Random Forest models to forecast final grades based on socio-economical conditions, attendance, and assignments.

Limitations:

- **Data Dependency:** The accuracy of the machine learning grade predictions depends directly on the volume and quality of historical and current data entered the system.
- **Financial Transactions:** The fee payment module will utilize sandbox payment gateways rather than live banking integrations.

- **Prediction Application:** Machine learning insights serve an advisory purpose; they identify areas for improvement based on data but do not guarantee final academic outcomes.

1.5 Development Methodology

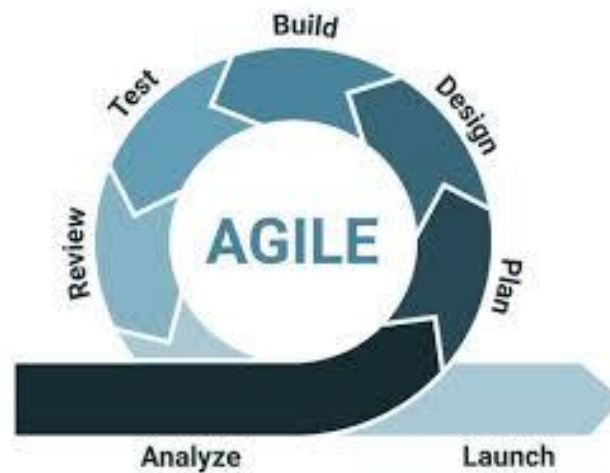


Figure 1.1 Agile methodology

1.6 Report Organization

Chapter 1: Introduction This initial chapter sets the foundation for the document by providing an overview of the project. It outlines the background, formally defines the problem statement, and establishes the primary objectives. Furthermore, it delineates the project's scope and limitations and introduces the development methodology adopted to guide the software engineering process.

Chapter 2: Background Study and Literature Review This chapter explores the theoretical framework and prior research relevant to the project. It details theories and terminologies associated with machine learning and digital academic management. Additionally, it examines similar existing systems and relevant literature to establish the context and necessity for the proposed solution.

Chapter 3: System Analysis This chapter provides a preliminary analysis of the system based on a structured methodology. It begins with requirement analysis, illustrating functional requirements through a use case diagram, and includes a feasibility study evaluating technical, operational, economic, and schedule constraints. The structural system modeling is then represented using Data Flow Diagrams (DFD Level 0 and DFD Level 1) for process modeling and Entity-Relationship (ER) diagrams for data modeling.

Chapter 4: System Design Building upon the structured analysis, this chapter translates system requirements into a technical blueprint. It details the database design through the relational database schema and outlines the system architecture, which is divided into three core modules. Additionally, this chapter provides algorithmic details, focusing specifically on the logic of the Random Forest machine learning model used for grade prediction.

Chapter 5: Implementation and Testing This chapter discusses the practical execution of the project. It lists the development tools, programming languages, and databases utilized, followed by the implementation details of the three system modules. The testing section is dedicated to unit testing to verify the functionality of individual components, and system testing.

Chapter 6: Conclusion and Future Recommendations The final chapter concludes the report by summarizing the project outcomes and evaluating the fulfillment of the initial objectives. It provides a final assessment of the system's effectiveness and offers recommendations for future enhancements.

2. BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

College Management Systems (CMS) is a centralized, web-based software application designed to digitize and manage the administrative and academic operations of an educational institution. Core components include student profile management, attendance tracking, assignment distribution, and fee processing. By utilizing multi-actor architecture, a CMS provides role-specific interfaces (dashboards) for administrators, teachers, and students.

Predictive Analytics Educational Data Mining (EDM) involves applying mathematical models to data generated by educational settings to understand and improve student learning outcomes. Predictive analytics in this context utilizes historical student data, such as attendance rates, periodic assessment scores, and assignment completion, to forecast future performance, specifically the final grade. This allows institutions to implement early warning systems for academically at-risk students.

The predictive component of this system relies on Supervised Machine Learning. In this algorithm is trained on a labeled dataset, meaning the input data is paired with the correct output (final grades). The model learns the mapping function from the inputs to the outputs to predict outcomes for unseen, future data.

Random Forest (RF) Algorithm Random Forest is an ensemble learning method for regression and classification tasks. It operates by constructing a multitude of decision trees during the training phase. For regression tasks, the output is the average prediction of the individual trees. This method utilizes "bagging" (bootstrap aggregating) to reduce overfitting and improve variance.

Dataset Specification To train and evaluate the Random Forest model, the system utilizes the "Student Performance Prediction" dataset sourced from Kaggle. This open-source dataset contains structured records encompassing student demographics, study habits, absent rate, and periodic academic assessments mapped to final academic outcomes. Utilizing this dataset allows the machine learning model to effectively learn the complex, non-linear relationships between behavioral or academic inputs and the target final grade. This ensures the predictive algorithm is well-calibrated before being applied to the real-time data generated within the proposed College Management System.

2.2. Literature Review

The integration of Machine Learning (ML) into educational systems has significantly enhanced the capability to predict student academic performance and identify at-risk learners at early stages.

Ensemble learning techniques, particularly Random Forest (RF), have demonstrated superior performance in academic prediction tasks. Agyemang et al. [1] reported that RF achieved higher predictive accuracy compared to alternative machine learning approaches due to its ability to capture nonlinear relationships and reduce overfitting through bootstrap aggregation. Similarly, Xu and Yang [4] concluded that RF outperforms traditional baseline models when interdependencies exist among predictive factors.

Beyond predictive algorithms, the foundational architecture of the management platform plays a critical role in educational administration. Literature concerning traditional web-based College Management Systems [5] highlights the operational efficiency gained by integrating academic and administrative modules. Studies on these architectures illustrate that centralizing student records, assignment workflows, and fee processing significantly reduces administrative overhead and improves information accessibility for both students and faculty. However, while these conventional platforms excel at data organization, they traditionally serve as static repositories and lack the predictive analytical capabilities required to proactively support student learning.

Bridging this gap, research in *IEEE Transactions on Learning Technologies* [3] highlights the use of predictive analytics to identify students at risk of dropout through early academic signals. AI-driven student success predictors discussed in *Computers in Human Behavior* [2] demonstrate high forecasting accuracy when sufficient and well-structured data is available, facilitating accurate systems and reliable academic interventions.

In the local context, existing Nepali platforms such as Paathshala [6] provide essential College Management System functionalities, including student profile management and attendance tracking. However, they do not integrate machine learning-based grade prediction or early warning analytics for identifying academically at-risk students. In many regional colleges, digital systems function merely as static data repositories without analytical capabilities. This creates a functional gap between data collection and the extraction of actionable academic insights, which the proposed system aims to resolve.

3. SYSTEM ANALYSIS

3.1 System Analysis

System Analysis is the process of examining and understanding a system to identify problems, gather requirements, and determine the best solution for developing or improving the system. It focuses on studying user needs, business processes, data flow, and system operations before the actual system design and implementation phase.

3.1.1 Requirement Analysis

i. Functional Requirements Functional requirements define the specific behaviors, operations, and information processing that the system must perform. The functional requirements of this system involve three primary actors: Administrators, Teachers, and Students.

- **User Registration & Authentication:** Secure account creation, role-based access control, and login mechanisms for Administrators, Teachers, and Students.
- **Administrator Module (Dashboard & Management):** * Register new students and manage comprehensive student profiles.
 - Manage teacher accounts and system access.
 - Access an overarching admin dashboard to view system-wide insights and data summaries at a glance.
- **Teacher Module (Academic Management):**
 - Input and update student attendance and periodic assessment marks (G1, G2).
 - Distribute assignments to respective classes or students.
 - Access a dedicated teacher dashboard to view individual student record summaries and track academic trends.
- **Student Module (Engagement & Finance):**
 - View distributed assignments and upload completed assignment files.
 - Process and track academic fee payments.

- View personal academic progress and attendance records.
- **Machine Learning Grade Prediction (G3):**
 - Predict the final grade (G3) of students based on historical and current data (attendance, G1, G2, extracurricular activity participation) utilizing the Random Forest algorithm.

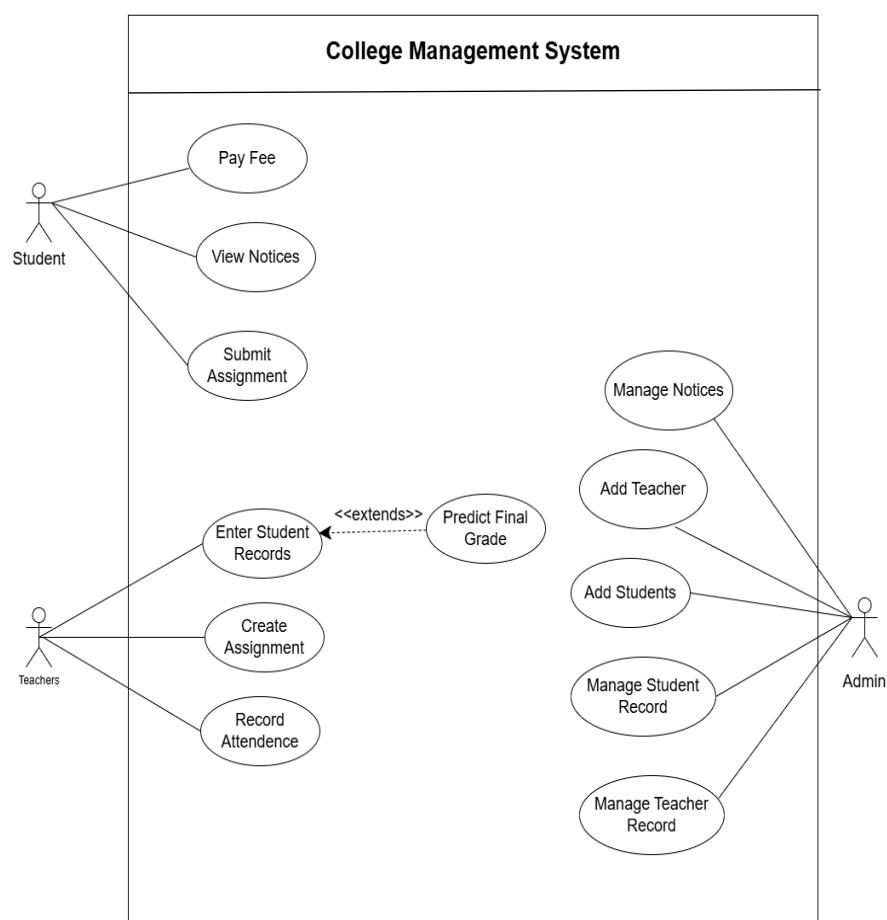


Figure 3.1 Use Case Diagram of College Management System

ii. Non-Functional Requirements Non-functional requirements specify the quality attributes, performance goals, and security parameters of the system.

- **Reliability and Data Integrity:** The system utilizes a safe, robust database structure to ensure academic and financial data is stored reliably without loss or corruption.
- **Performance and Speed:** The web application loads pages quickly, and the machine learning grade predictions must be generated and displayed in under 10 seconds. Furthermore, the system must smoothly handle concurrent sessions (e.g., 50+ students and multiple teachers accessing the system simultaneously).
- **Security:** Password is securely hashed, and strict role-based authentication must be enforced so that users can only access data authorized for their specific role.
- **Availability:** The web system maintains a 99% uptime during standard academic semesters to ensure constant accessibility for assignments and administrative tasks.
- **Usability:** The interface is intuitive, ensuring that users with minimal technical expertise can easily navigate dashboards, upload assignments, or input grades.

3.1.2 Feasibility Analysis

i. Technical Feasibility The project relies on well-established, open-source technologies with extensive documentation. React.js will handle the responsive frontend web interface, while Node.js with Express manages the backend logic and API routes. PostgreSQL is utilized to ensure reliable data storage. Predictive modeling is executed using the Scikit-learn in Python to deploy the Random Forest algorithm. Given the availability of these tools and the team's familiarity with them, the project is highly technically feasible.

ii. Operational Feasibility The web-based system is designed to seamlessly integrate into daily college operations. Because it features a user-friendly interface, it does not require technical training for staff or students. Teachers can input records effortlessly, students can manage assignments and fees intuitively, and administrators can oversee operations through centralized dashboards. This ensures smooth operational adoption within a campus environment.

iii. Economic Feasibility The development of this system is economically feasible. There is no requirement for specialized external hardware, as standard student and staff devices (laptops, smartphones) are enough. Furthermore, the core development technologies (React, Node.js, Python) are open-source and free to use. Model training utilizes the open-source Kaggle dataset.

iv. Schedule Feasibility The project utilizes an Agile development methodology, breaking the system down into manageable sprints. By prioritizing core modules first (student management and authentication), followed by secondary modules (assignments, fees, dashboards) and finally the machine learning integration, the workload is evenly distributed. This structured, time-bound approach ensures that the entire project is completed, tested, and deployed within the designated timeframe of the academic semester.

3.1.3 Analysis

To comprehensively analyze the system's operational mechanisms, a structured methodology was adopted. This involves process modeling to map the flow of information and data modeling to structure the underlying database.

Process Modelling using DFD (Data Flow Diagram) Data Flow Diagrams are utilized to represent the flow of information within the College Management System.

DFD Level 0 (Context Diagram):

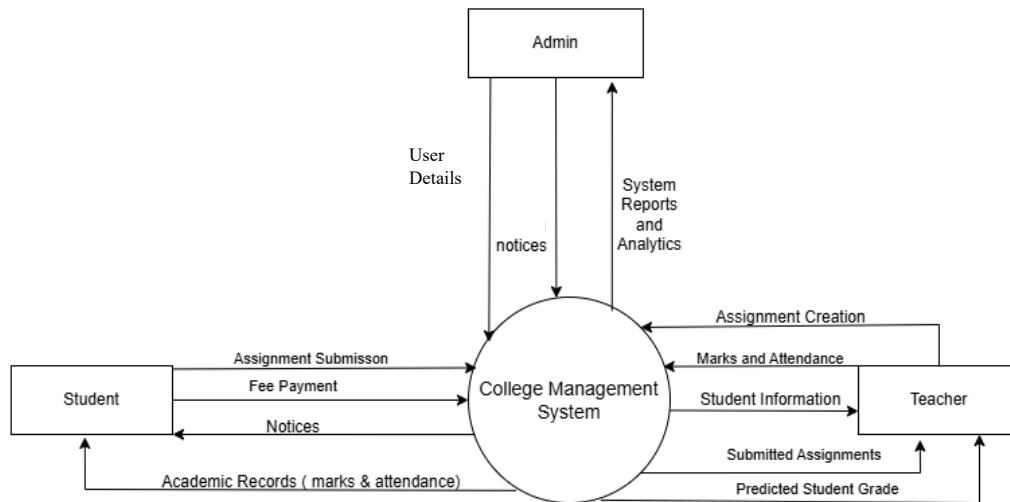


Figure 3.2 Context Diagram of College Management System

This diagram illustrates the entire system as a single process interacting with external entities, namely the Administrator, Teacher, and Student. It shows the high-level inputs (e.g., grade entry, assignment uploads, fee payments) and outputs (e.g., dashboards, grade predictions, insights) flowing in and out of the centralized system.

DFD Level 1:

This diagram breaks down the main system into detailed sub-processes, mapping exactly how data moves between the User Authentication module, the Academic Record module, the Assignment & Fee modules, and the Machine Learning Prediction Engine. It clarifies how data stores (like the student database) interact with these individual processes.

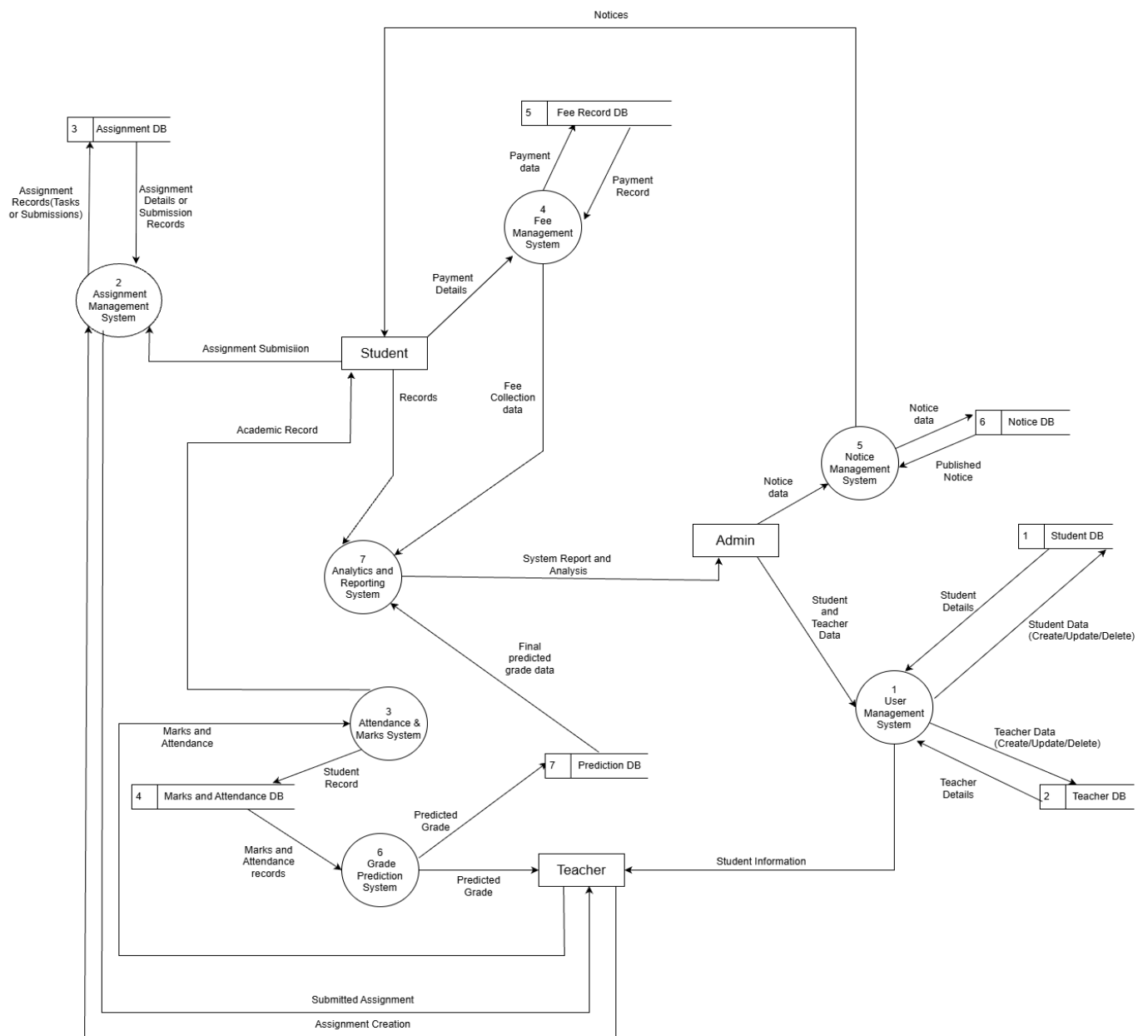


Figure 3.3 DFD level 1 of College Management System

Entity Relationship Diagram

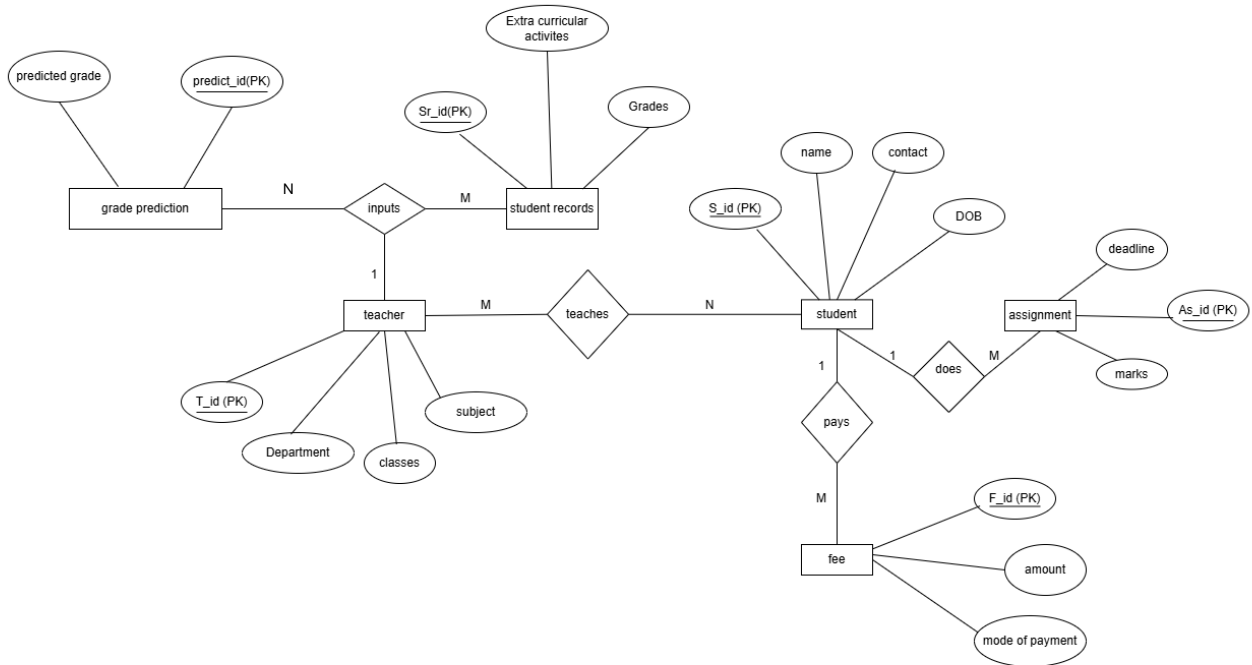


Figure 3.4 Entity-Relationship Diagram

Data Modelling using ER Diagrams

The Entity-Relationship (ER) diagram provides a logical blueprint of the system's database architecture. It defines the core entities, their specific attributes, and the cardinal relationships connecting them, ensuring that data is stored efficiently without redundancy. Based on the system's structural design, the primary components are mapped as follows:

Core Entities and Attributes:

- **Student:** Stores personal information with attributes including S_id (Primary Key), name, contact, and DOB (Date of Birth).
- **Teacher:** Stores faculty details with attributes including T_id (Primary Key), Department, classes, and subject.

- Student Records: Stores academic tracking data with attributes like Sr_id (Primary Key), Grades, and Extracurricular activities.
- Grade Prediction: Stores the machine learning outputs with attributes including predict_id (Primary Key) and the final predicted grade.
- Assignment: Stores task details with attributes including As_id (Primary Key), deadline, and marks.
- Fee: Stores financial transactions with attributes including F_id (Primary Key), amount, and mode of payment.

4. SYSTEM DESIGN

4.1 Design

This phase translates the requirement specifications and preliminary system analysis into a concrete technical blueprint. Following a structured analysis approach, the design phase focuses on transforming the logical ER diagrams into relational database tables, outlining the user interfaces, and defining system inputs and outputs.

4.1.1 Database Design: Transformation of ER to Relations and Normalizations

The logical Entity-Relationship (ER) models are transformed into relational schemas. To eliminate data redundancy and ensure data integrity, the tables are normalized up to the Third Normal Form (3NF). The database architecture is divided into three core functional modules:

Module for Administration & Setup This module handles the core configuration, user roles, and system-wide announcements.

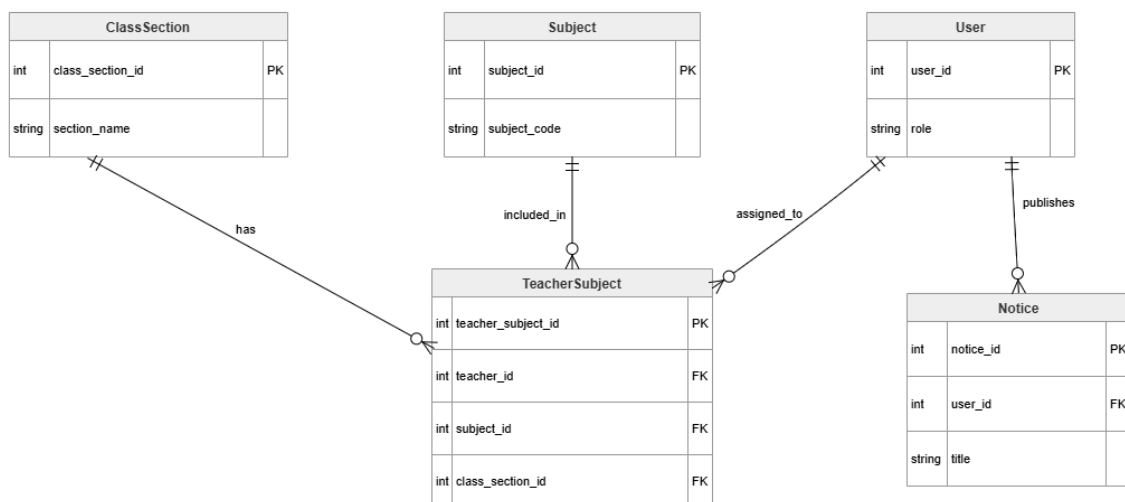


Figure 4.1 Relational Schema Diagram of administration and setup module

- **User Relation:** Stores authentication and role data. The primary key is `user_id`, with attributes defining the user's role (Admin, Teacher, or Student).
- **Subject & ClassSection Relations:** The Subject table stores course details (`subject_id` PK, `subject_code`), while ClassSection manages grouping (`class_section_id` PK, `section_name`).
- **TeacherSubject Relation:** Acts as a junction table to map which teacher instructs which subject in which section. It uses `teacher_subject_id` as the primary key and holds foreign keys linking to User, Subject, and ClassSection.
- **Notice Relation:** Stores system announcements, containing `notice_id` (PK), title, and a foreign key (`user_id`) linking back to the user who published it.

Module 2: Academic Performance & Daily Tracking

This module manages the core academic workflows, including assignments, attendance, and machine learning outputs.

- **Student Relation:** Stores individual student records, utilizing `student_id` (PK) and a foreign key to `class_section_id`.
- **Assignment & Submission Relations:** The Assignment table is linked to the teacher who created it via a foreign key (`teacher_subject_id`). When a student completes work, it is recorded in the Submission table, linking the `assignment_id` and `student_id` via foreign keys.
- **Attendance Relation:** Tracks daily presence, containing an `attendance_id` (PK) and foreign keys linking the specific `student_id` to the respective `teacher_subject_id`.
- **GradePrediction Relation:** Stores the machine learning outputs. It uses `prediction_id` (PK) and links to the `student_id`, storing the specific `predicted_grade`.

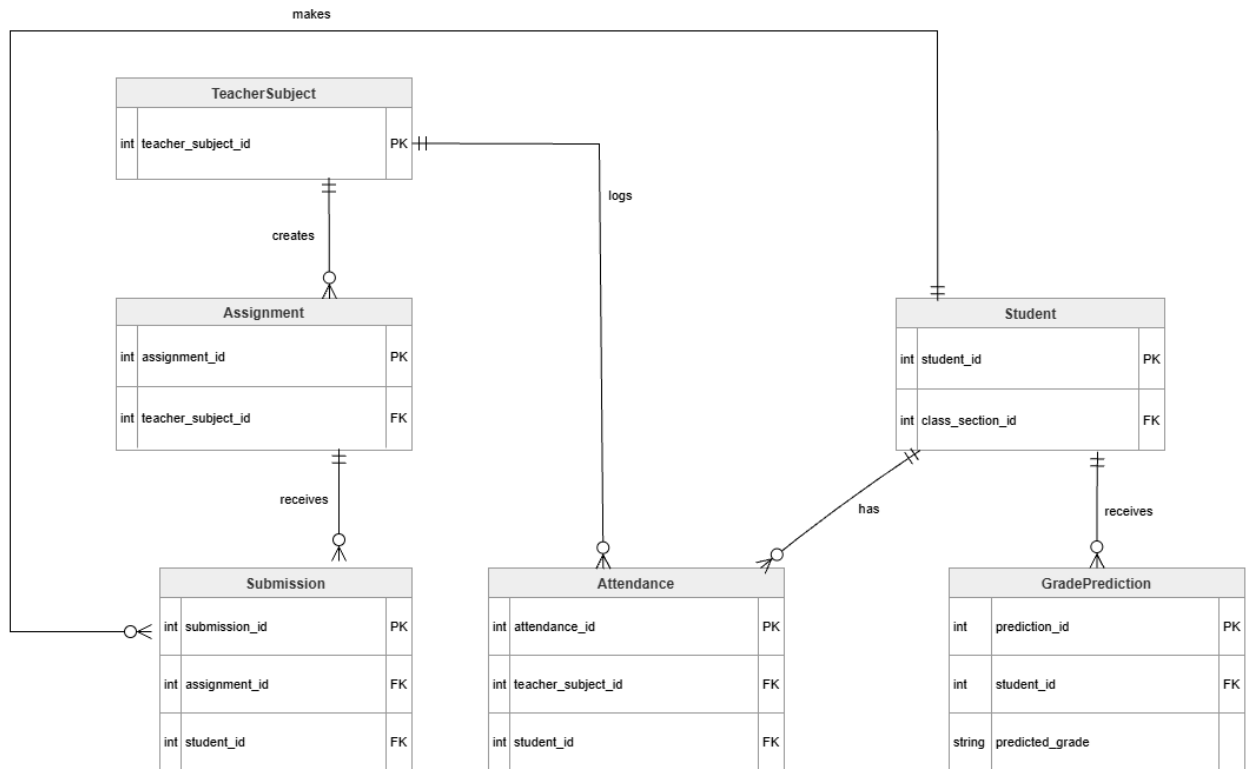


Figure 4.2 Relational Schema for Academic Performance and Daily Tracking

Module 3: Finance & Billing

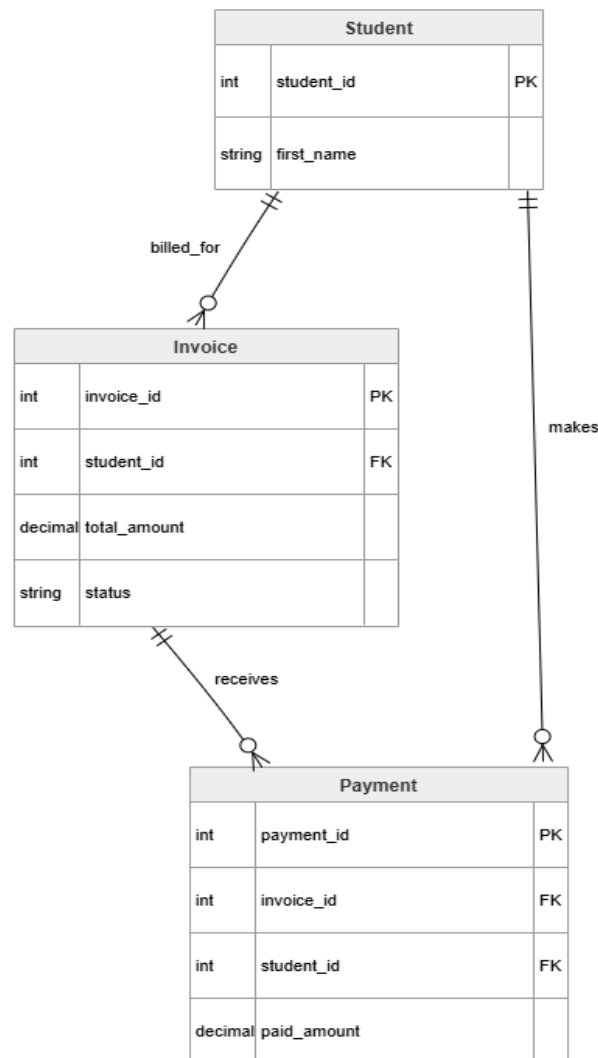


Figure 4.3 Relational Schema for Fee module

This module streamlines the financial operations between the college and the students.

- **Invoice Relation:** Generates the fee bills for students. It contains `invoice_id` (PK), `total_amount`, billing status, and a foreign key linking to the respective `student_id`.
- **Payment Relation:** Tracks transactions made against invoices. It contains `payment_id` (PK), `paid_amount`, and foreign keys linking back to both the `invoice_id` and the `student_id`.

4.2 Algorithm Details

Random Forest

The prediction service uses a Random Forest classifier to estimate the final student grade (A, B, C, D, F) based on academic and behavioral inputs. Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

The Random Forest builds multiple decision trees, where each tree is trained on a random subset of the dataset. The final prediction is obtained through majority voting among all trees:

$$\hat{y} = \text{mode}(h_1(x), h_2(x), \dots, h_N(x))$$

Where:

- $h_i(x)$ = prediction of the i-th decision tree
- N = number of trees (300 in this model)
- $\hat{y}$ = final predicted class

The model also computes prediction confidence using class probabilities:

$$P(y = c \mid x) = \frac{1}{N} \sum_{i=1}^N I(h_i(x) = c)$$

Where:

- c = class label (A, B, C, D, F)
- $I(\cdot)$ = indicator function

Each decision tree is constructed by minimizing node impurity using the Gini Index:

$$\text{Gini}(D) = 1 - \sum_{i=1}^c p_i^2$$

Model Configuration

- Number of trees: 300

- Random state: 42 (ensures reproducibility)
- Class weight: balanced (to handle class imbalance)

Feature Importance Analysis

We selected these 7 features after performing a feature importance analysis on the full UCI student performance dataset. The Random Forest model's importance scores consistently identified academic performance indicators (G1, G2), attendance (absences), and contextual/behavioral factors (extracurricular activities, Mjob, Fjob, traveltime) as the most predictive variables for final grade (G3).

Features with low importance or redundant predictive value were removed to simplify the model. This reduction improves computational efficiency, reduces overfitting risk, and makes the system easier to maintain, while still preserving the primary factors influencing student performance.

This approach achieves a strong balance between model accuracy and simplicity, ensuring practical usability in a real-world educational environment.

5. IMPLEMENTATION

5.1 Implementation

The system was implemented using appropriate CASE tools, programming languages, and database platforms to ensure efficiency and reliability. Each module and function was systematically developed to support smooth integration and testing.

5.1.1 Tools Used

Programming Languages

- JavaScript (Node.js): Used for the backend API to handle routing, business logic, and database access.
- JavaScript/JSX (React): Used for the frontend UI to build reusable components and manage client-side state.
- Python: Used for the ML microservice to train, serialize, and serve the prediction model.

Frontend

- React: Component-based UI library for building interactive pages like admin, teacher, and student dashboards.
- Vite: Fast development server and build tool for bundling the React app.

Backend

- Node.js (Express): REST API server to manage authentication, CRUD operations, and integration with ML service.
- Prisma ORM: Type-safe database access layer to map PostgreSQL tables to JavaScript objects.

Database

PostgreSQL: Relational database to store users, classes, fees, attendance, and academic records.

Machine Learning (ML)

- FastAPI: Python web framework to expose ML prediction endpoints.
- scikit-learn (Random Forest): Model algorithm used for student performance prediction.
- pandas, numpy: Data cleaning and feature engineering for training.
- joblib: Saves and loads trained models efficiently.

Development / CASE Tools

- Visual Studio Code: IDE for coding, debugging, and running scripts.
- Git: Version control for tracking changes and collaborating.
- Draw.io: Used to create architecture and ER diagrams for documentation.
-

Package Managers

- npm: Manages frontend and backend JavaScript dependencies.
- pip: Manages Python libraries for the ML service.

5.1.2 Implementation Details of Modules

Authentication & Identity

- Mechanism:
Implements stateless authentication using JWT tokens signed with JWT_SECRET and configured with a 7-day expiration period.
- Access
Role-based authorization is handled through requireAuth(role?) middleware, which validates the Authorization: Bearer <token> header, verifies token integrity, and injects authenticated user information into req.user.
- Endpoints:

- POST /api/auth/login — Validates user credentials using bcrypt.compare() password hashing.
- GET /api/auth/me — Retrieves authenticated user details using req.user.userId.

Attendance Analytics Engine

- **Algorithm:**
Uses single-pass $O(n)$ in-memory aggregation with maps keyed by studentId and class attributes (batch, faculty, section) to improve processing efficiency and reduce database overhead.
- **Boundaries:**
Monthly attendance analytics are filtered using UTC-based date ranges through getCurrentMonthRangeUtc().
- **Formula:**
Attendance percentage is calculated using:
 Attendance Percentage = $\text{round}\left(\frac{\text{Present}}{\text{Total}} \times 100\right)$

Notice Engine

- **Distribution**
The POST /api/notices endpoint dynamically resolves recipients based on three scopes:
 - ALL — All students and teachers
 - CLASS — Targeted class and assigned teacher
 - INDIVIDUAL — Explicitly selected users
- **Tracking:**
The POST /api/notices/:id/read endpoint updates individual readAt timestamps, enabling real-time notice read tracking and analytics.

Predictive ML Pipeline

API Logic

- Endpoint: POST /api/predict-grade
- Teachers must provide subjectId.
- Admins use average marks across all subjects.

Attendance Handling

- Attendance is converted into an absence signal.
- If records exist, student/class absence is calculated.
- If absent data is missing, high absence is assumed.
- Used to adjust final grade prediction.

ML Service Integration

- Connected via ML_SERVICE_URL.
- Uses /predict endpoint.
- Returns letter grade.
- Backend integrates ML output into final result.

5.2 Testing

Testing is a critical phase of the Software Development Life Cycle (SDLC) used to ensure that the "Administration" and "Academic" modules function according to the requirements. A Bottom-Up testing strategy was employed, starting with individual code components and ending with full user-level workflows.

5.2.1 Test Cases for Unit Testing

Test Case Id 1: Unit Test UT-1: Prediction System Testing

Step	Test Steps	Test Data	Expected Result	Actual Result (short)	Status
1	Trigger FastAPI startup event	ModelPath: /model/random_forest.pkl	Application must load the model bundle from disk during startup.	Model loaded at startup	Pass

2	Start service with missing model file	ModelPath: /model/random_forest.pkl (file removed)	Application must fail startup and log that the model file is missing.	Startup failed, file missing	Pass
3	Predict with valid input	POST /predict with full valid JSON body	Service must return a predicted grade and a confidence value.	Returned grade + confidence	Pass
4	Predict with missing required field	POST /predict missing G1	Service must reject the request and explain the missing feature.	Rejected, explained missing field	Pass
5	Predict with edge values	POST /predict with min/max valid values	Service must accept the request and return a valid prediction.	Accepted, valid prediction	Pass
6	Confirm feature order mapping	POST /predict with known values	Service must map features in the expected order before prediction.	Features mapped correctly	Pass

Test Case Id 2: Unit Test UT-2: Authentication

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Health endpoint	Call health endpoint	200, server running message	Server responded OK	Pass
Admin login valid	POST login with admin email/password	200, token + user returned	Logged in, token returned	Pass
Login wrong password	POST login with correct email, wrong password	401, "Invalid credentials"	Login rejected	Pass
Get current user with token	GET current-user with token	200, user returned	User returned	Pass
Get current user missing token	GET current-user without token	401, "Missing token"	Rejected: missing token	Pass

Test Case ID 3: Unit Test UT-3: User Management

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Create teacher valid	Admin creates teacher (name/email/password)	Success, teacher created	Teacher created	Pass
Create teacher duplicate email	Admin uses existing email	409, "Email already exists"	Conflict returned	Pass
Create teacher invalid input	Admin submits invalid email/password	400, validation errors	Validation failed	Pass
Non-admin blocked from create	Teacher tries to create teacher	403 Forbidden	Forbidden	Pass

Update teacher valid	Admin updates teacher fields	Success, updated user	Teacher updated	Pass
Delete teacher valid	Admin deletes teacher	Success, returned id	Teacher deleted	Pass

Test Case Id 4: Unit Test UT-4: Student management and class access

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Create student valid	Admin creates student with required fields	Success, student created	Student created	Pass
Create student duplicate roll	Admin creates student with same roll/class	409, rollNumber conflict	Conflict returned	Pass
Create student invalid input	Admin submits invalid/missing fields	400, validation errors	Validation failed	Pass
List students (admin)	Admin requests students list	200, students returned	Students listed	Pass
Teacher lists assigned class	Teacher requests students for their class	200, class students returned	Students listed for class	Pass

Test Case Id 5: Unit Test UT-5: Notices

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Create notice (ALL)	Admin posts notice scope ALL	Success, notice created	Notice created	Pass
Create CLASS notice missing fields	Admin posts CLASS notice without class info	400, missing class data	Rejected: missing class info	Pass
Create INDIVIDUAL notice missing recipients	Admin posts INDIVIDUAL without recipients	400, recipientIds required	Rejected: missing recipients	Pass
Get notices for user	User fetches notices	200, notices returned	Notices returned	Pass
Mark notice read valid	User marks notice as read	Success, marked true	Marked read	Pass
Mark notice invalid id	User marks invalid id	400, invalid id	Rejected: invalid id	Pass

Test Case Id 6: Unit Test UT-6: Assignment and Submission

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Teacher creates assignment	Teacher posts assignment with class info	Success, assignment created	Assignment created	Pass
Student blocked from create	Student tries to create assignment	403 Forbidden	Forbidden	Pass
Create assignment missing fields	Teacher posts incomplete assignment	400, validation error	Validation failed	Pass

Student submits assignment valid	Student submits note/file to their assignment	Success, submission saved	Submission saved	Pass
Student submits to wrong class	Student submits to assignment for other class	403, not for your class	Rejected: wrong class	Pass
Teacher views submissions	Teacher fetches submissions for own assignment	200, submissions returned	Submissions returned	Pass

5.2.2. Test Cases for System Testing

System testing validates the complete integrated software. It ensures that different modules (like Users and Class Sections) work together to complete a business process

Test Case Id 7: System Test ST-1: Admin End to End flow

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Admin login success	UI Login: admin credentials	Redirect to /admin and workspace loads	Logged in, redirected to admin	Pass
Admin login failure	wrong password	Error shown and stay on login page	Login rejected	Pass
Load dashboard data	Admin dashboard open	Dashboard widgets and charts render	Dashboard loaded	Pass
Create teacher account	Admin → Teachers → create valid user	Teacher appears in list; success notice	Teacher created	Pass
Update teacher profile	Edit name/email/password	Teacher list updates; success notice	Teacher updated	Pass
Delete teacher	Delete a teacher row	Teacher removed; success notice	Teacher deleted	Pass
Create student	Admin → Students → valid data	Student appears; success notice	Student created	Pass

Filter students by class	Select batch/faculty/section	Students list filtered correctly	Filter applied	Pass
Create subject	Academics → add subject	Subject listed in subjects table	Subject created	Pass
Assign teacher to subject	Subject assignment valid	Assignment saved and visible	Assignment saved	Pass
Assign class teacher	Class teacher assignment valid	Assignment saved; teacher sees class	Class assigned	Pass
View attendance summary	Attendance Summary filter class	Summary table loads correctly	Summary displayed	Pass
Create notice (ALL)	Notices → scope ALL	Notice delivered to teachers + students	Notice delivered	Pass
View prediction history	Predictions list (optional filter)	Predictions list loads with grades/confidence	Predictions loaded	Pass

Test Case Id 8: System Test ST-2: Teacher End to End flow

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Teacher login success	UI Login: teacher credentials	Redirect to /teacher and workspace loads	Logged in, redirected to teacher	Pass
View assigned classes	Teacher dashboard	Only assigned classes appear	Assigned classes shown	Pass
View class students	Students page + class filter	Students list loads for selected class	Students loaded	Pass
Take attendance	Attendance → class/date → submit	Attendance saved; success notice	Attendance saved	Pass
Edit attendance	Pick already-taken date	Attendance updates successfully	Attendance updated	Pass
Add marks	Marks → student → save	Marks saved; success notice	Marks saved	Pass
Run prediction	Prediction → select student	Predicted grade shown and stored	Prediction saved	Pass

Create assignment	Assignments → upload + submit	Assignment appears in list	Assignment created	Pass
View submissions	Open assignment submissions	Submissions list loads	Submissions loaded	Pass
Notices inbox	Mark notice as read	Read status updates	Marked read	Pass

Test Case Id 9: System Test ST-3: Student End to End flow

Test Case	Test Data	Expected Result	Actual Result	Pass/Fail
Student login success	UI Login: student credentials	Redirect to /student and dashboard loads	Logged in, redirected to student	Pass
Load notices	Student dashboard	Notices list and counts load	Notices loaded	Pass
Mark notice read	Click “mark read”	Notice stays read on refresh	Marked read	Pass
Load assignments	Assignments page	Class assignments list loads	Assignments loaded	Pass
Submit assignment (note only)	Note + submit	Submission saved and visible	Submission saved	Pass
Submit assignment (with file)	Attach file + submit	File upload succeeds; attachment shown	File uploaded	Pass
View attendance history	Attendance/summary page	Attendance history renders	History displayed	Pass
View latest marks	Marks page	Latest marks displayed	Marks displayed	Pass

6. CONCLUSION

6.1 Conclusion

This College Management System (CMS) was successfully implemented. It provides a real-world solution for academic administration by integrating a modern web client. The system created distinct roles (admin, teacher, student) and delivers complete workflows for attendance, academics, notices, assignments, class-teacher mapping, fee tracking, and dashboard analytics. In addition, the project includes an ML service for student performance analysis and prediction, showing how data-driven insights can be integrated into institutional decision-making. This combination of transactional features (records, workflows, authentication) and analytical capabilities (prediction, summaries) demonstrate a full-stack, end-to-end system aligned with the needs of modern educational institutions. The modular architecture, and reusable components make the solution scalable and maintainable, while also leaving clear space for future extensions such as automated reporting, advanced analytics, and integration with external platforms. Overall, the project meets its objectives.

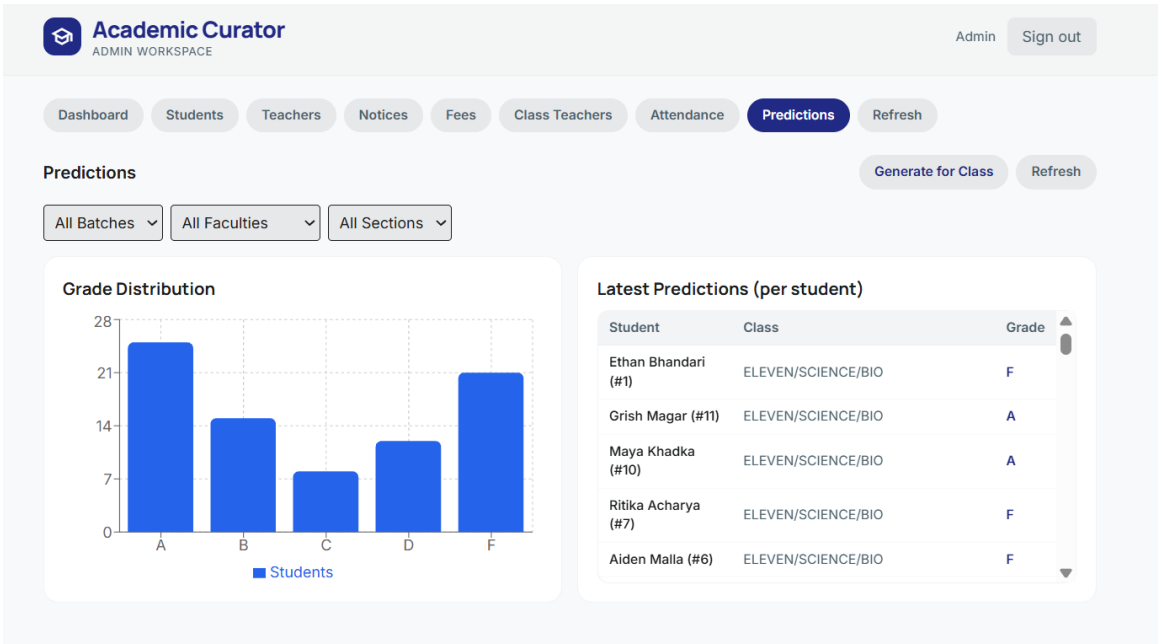
6.2 Future Recommendation

To strengthen the efficiency and user experience of the College Management System, several forward-looking enhancements can be introduced. First, integrating image processing technology will allow assignments to be checked automatically, reducing the manual workload on teachers while ensuring consistency. Second, developing a dedicated mobile application will make the system more accessible to students, staff, and administrators on the go. Third, implementing analytics and interactive dashboards will provide long-term insights into academic performance, resource utilization, and institutional growth. Additionally, offering a parent/guardian dashboard with SMS integration for fee updates will improve transparency and communication. Finally, embedding video lectures and class recordings

7. REFERENCES

- [1] S. Agyemang, E. O. Appiah, and E. A. Yirenkyi, "Predicting Students' Academic Performance Via Machine Learning Techniques," ScholarWorks@UTRGV, 2023.
- [2] J. Xu and H. Yang, "A comparative study of Random Forest and Multiple Linear Regression models in predicting student academic performance," Expert Systems with Applications, vol. 185, p. 115629, 2022.
- [3] W. A. Sajid, N. Mirzah, F. M. Mustafa, and Y. Shabala, "Educational Institution Management Information System," in 2024 35th Conference of Open Innovations Association (FRUCT), IEEE, 2024, pp. 625-632.
- [4] T. Gray, C. Perkins, and R. Lewis, "Using learning analytics to predict student retention in higher education," IEEE Transactions on Learning Technologies, vol. 14, pp. 115-128, 2021.
- [5] M. Shoaib, N. Sayed, J. Singh, J. Shafi, S. Khan, and F. Ali, "AI student success predictor: Enhancing personalized learning in campus management systems," Computers in Human Behavior, vol. 158, p. 108301, Sep. 2024.
- [6] Paathshala, "School-College Management System Features," 2025. [Online]. Available: <https://paathshala.com.np/>

8. APPENDICES



Academic Curator

ADMIN WORKSPACE

Admin

Sign out

Dashboard

Students

Teachers

Notices

Fees

Class Teachers

Attendance

Predictions

Refresh

Create Notice

Title

Body

Scope: All (students & teachers)

Create Notice

Past Notices

Refresh

holiday tommorow

holiday

Scope: ALL • All users • 5/10/2026, 7:07:46 PM

Read: 0/18

hello

dog

Scope: ALL • All users • 5/1/2026, 10:46:15 AM

Read: 0/18

holiday tommorow

this is to tell you that tommrow will be a holiday

Scope: ALL • All users • 5/1/2026, 10:33:35 AM

Read: 1/18

Academic Curator

ADMIN WORKSPACE

AdminSign out

Dashboard

Students

Teachers

Notices

Fees

Class Teachers

Attendance

Predictions

Refresh

GRADE 11

DEPARTMENT ALL

SUBJECT ALL

SEARCH

Search by name or email

Clear Filters

+ Add Teacher

Showing: Grade 11 | Department: All | Subject: All

Teacher Management

Showing 14 teachers

Grade 11

SCIENCE

Roshan Lama roshan.lama@campus.local	SUBJECT(S) Computer Science, English	Edit Delete
Sushil Adhikari sushil.adhikari@campus.local	SUBJECT(S) Mathematics	Edit Delete
Bikram Shrestha bikram.shrestha@campus.local	SUBJECT(S) Mathematics	Edit Delete
Dipesh Poudel Class Teacher 11/SCIENCE/BIO dipesh.poudel@campus.local	SUBJECT(S) Physics	Edit Delete
Anjana Rai Class Teacher 11/SCIENCE/BIO anjana.raib@campus.local	SUBJECT(S) Biology	Edit Delete

Academic Curator

ADMIN WORKSPACE

AdminSign out

Dashboard

Students

Teachers

Notices

Fees

Class Teachers

Attendance

Predictions

Refresh

Attendance Overview

Use the month selector to navigate chart data.

May2026

TOTAL STUDENTS

84

ATTENDANCE % (MAY 2026)

72%

PRESENT (LATEST DAY IN MAY 2026)

63

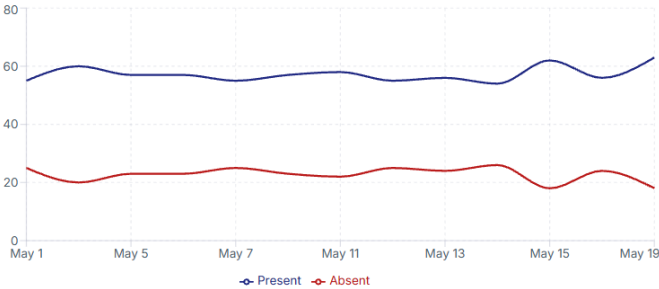
TOTAL FACULTY

18

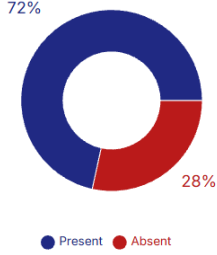
+ Add New Student

+ Register Teacher

Global Daily Attendance Trend (May 2026)



Present vs Absent (May 2026)



35